Unit-V

Backtracking and Branch and Bound

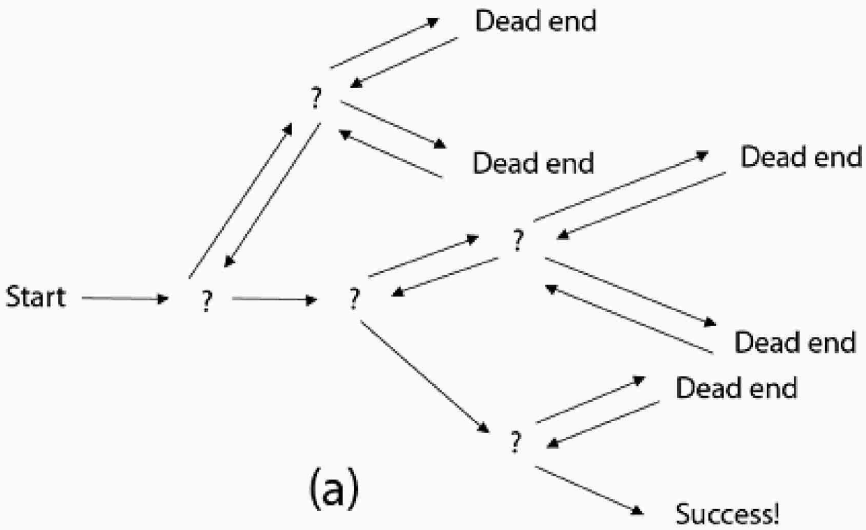
Syllabus- Backtracking: The General Method, the 8-Queens Problem, Sum of Subsets, Graph Coloring, Hamiltonian Cycles.
Branch and Bound: General method, applications - Travelling sales person problem,0/1 knapsack problem- LC Branch and Bound solution, FIFO Branch and Bound solution.

Backtracking General Method:

The Backtracking is an algorithmic-method to solve a problem with an additional way. It uses a recursive approach to explain the problems. We can say that the backtracking is needed to find all possible combination to solve an optimization problem.

Backtracking is a systematic way of trying out different sequences of decisions until we find one that "works."

- Each non-leaf node in a tree is a parent of one or more other nodes (its children)
- Each node in the tree, other than the root, has exactly one parent



- It is convenient to implement this kind of processing by constructing a tree of choices being made, called the ***state-space tree***.
- Its root represents an initial state before the search for a solution begins. The nodes of the first level in the ree represent the choices made for the first component of a solution, the nodes of the second level represent the choices for the second component, and so on.
- A node in a state-space tree is said to be ***promising*** if it corresponds to a partially constructed solution that may still lead to a complete solution; otherwise, it is called ***nonpromising***.
- Leaves represent either nonpromising dead ends or complete solutions found by the algorithm.
- In the majority of cases, a statespace tree for a backtracking algorithm is constructed in the manner of depthfirst search**(DFS).**

What is a state-space tree?
The tree construted to implement backtracking with the choices for the components is called the state-space tree. Its root represents initial state before the search for a solution and the nodes at each level represent the choices made for the corresponding component of a solution.

Terminology:
Answer states: These are solution states where the path from the root to the leaf defines the solution of the problem.
Live node is a node that has been generated but whose children have not yet been generated.
E-node is a live node whose children are currently being explored. In other words, an E-node is a node currently being expanded.
Dead node is a generated node that is not to be expanded or explored any further. All children of a dead node have already been expanded.

1. n-Queens Problem:

The problem is to place n queens on an $n \times n$ chessboard so that no two queens attack each other by being in the same row or in the same column or on the same diagonal.
For $n = 1$, the problem has a trivial solution, and it is easy to see that there is no solution for $n = 2$ and $n = 3$.
a) 4-Queens Problem: So let us consider the four-queens problem and solve it by the backtracking technique. Since each of the four queens has to be placed in its own row, all we need to do is to assign a column for each queen on the board presented in Figure 12.1.
We start with the empty board and then place queen 1 in the first possible position of its row, which is in column 1 of row 1. Then we place queen 2, after trying unsuccessfully columns 1 and 2, in the first acceptable position for it, which is square (2, 3), the square in row 2 and column 3. This proves to be a dead end because there is no acceptable position for queen 3. So, the algorithm backtracks and puts queen 2 in the next possible position at (2, 4). Then queen 3 is placed at (3, 2), which proves to be another dead end. The algorithm then backtracks all the way to queen 1 and moves it to (1, 2). Queen 2 then goes to (2, 4), queen 3 to (3, 1), and queen 4 to (4, 3), which is a solution to the problem. The state-space tree of this search is shown in Figure 12.2.
If other solutions need to be found (how many of them are there for the fourqueens problem?), the algorithm can simply resume its operations at the leaf at which it stopped. Alternatively, we can use the board's symmetry for this purpose.

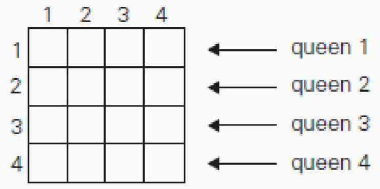


FIGURE 12.1 Board for the four-queens problem.

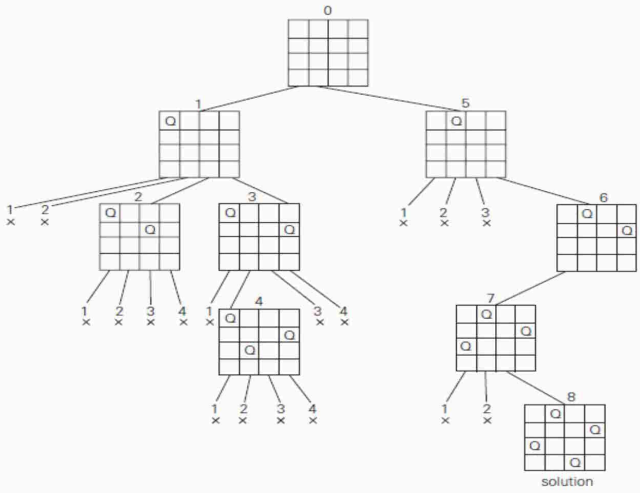


FIGURE 12.2 State-space tree of solving the four-queens problem by backtracking. × denotes an unsuccessful attempt to place a queen in the indicated column. The numbers above the nodes indicate the order in which the nodes are generated.

b) 8-Queens Problem:

One possible solution for 8 queens problem is shown in fig:

	1	2	3	4	5	6	7	8
1				q_1				
2						q_2		
3								q_3
4		q_4						
5							q_5	
6	q_6							
7			q_7					
8					q_8			

Thus, the solution **for** 8 -queen problem **is** (4, 6, 8, 2, 7, 1, 3, 5).

2. Sum of Subsets or Subset-Sum Problem

We consider the ***subset-sum problem***, find a subset of a given set $A = \{a1, \ldots, an\}$ of n positive integers whose sum is equal to a given positive integer d.

It is convenient to sort the set's elements in increasing order. So, we will assume that $a1 < a2 < \ldots < an$.

The state-space tree can be constructed as a binary tree like that in Figure 12.4 for the instance $A = \{3, 5, 6, 7\}$ and $d = 15$.

The root of the tree represents the starting point, with no decisions about the given elements made as yet. Its left and right children represent, respectively, inclusion and exclusion of $a1$ in a set being sought.

Similarly, going to the left from a node of the first level corresponds to inclusion of $a2$ while going to the right corresponds to its exclusion, and so on. Thus, a path from the root to a node on the ith level of the tree indicates which of the first i numbers have been included in the subsets represented by that node.

We record the value of s, the sum of these numbers, in the node. If s is equal to d, we have a solution to the problem. We can either report this result and stop or, if all the solutions need to be found, continue by backtracking to the node's parent. If s is not equal to d, we can terminate the node as nonpromising if either of the following two inequalities holds:

$$s + a_{i+1} > d \quad \text{(the sum } s \text{ is too large)},$$
$$s + \sum_{j=i+1}^{n} a_j < d \quad \text{(the sum } s \text{ is too small)}.$$

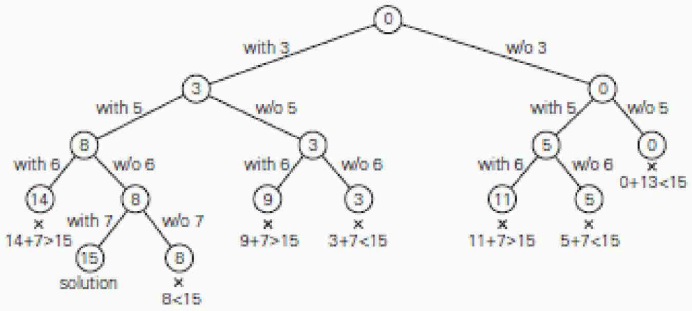


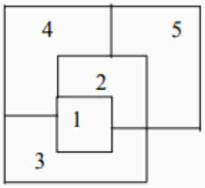
FIGURE 12.4 Complete state-space tree of the backtracking algorithm applied to the instance $A = \{3, 5, 6, 7\}$ and $d = 15$ of the subset-sum problem. The number inside a node is the sum of the elements already included in the subsets represented by the node. The inequality below a leaf indicates the reason for its termination.

The Time-complexity is $O(2^n)$ i.e. Exponential time, where n is the number of elements.

3. GRAPH COLORING PROBLEM

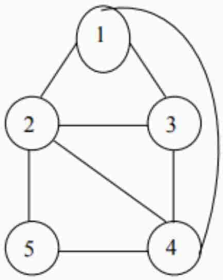
- Let 'G' be a graph and 'm' be a given positive integer. If the nodes of 'G' can be colored in such a way that no two adjacent nodes have the same color. Yet only 'M' colors are used. So it's called M-color ability decision problem.
- The graph G can be colored using the smallest integer 'm'. This integer is referred to as chromatic number of the graph.
- A graph is said to be planar iff it can be drawn on plane in such a way that no two edges cross each other.
- Suppose we are given a map then, we have to convert it into planar. Consider each and every region as a node. If two regions are adjacent then the corresponding nodes are joined by an edge.

Consider a map with five regions and its graph.

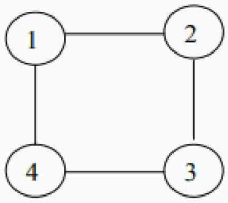


1 is adjacent to 2, 3, 4.
2 is adjacent to 1, 3, 4, 5
3 is adjacent to 1, 2, 4
4 is adjacent to 1, 2, 3, 5
5 is adjacent to 2, 4

Planar graph representation of map



Example:



N= 4
M= 3

Adjacency Matrix:

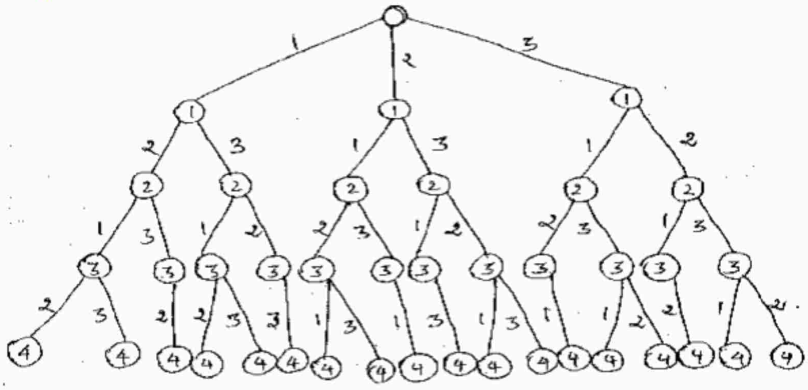
$$\begin{vmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{vmatrix}$$

→ Problem is to color the given graph of 4 nodes using 3 colors.

→Node-1 can take the given graph of 4 nodes using 3 colors.

→ The state space tree will give all possible colors in that, the numbers which are inside the circles are nodes, and the branch with a number is the colors of the nodes.

State Space Tree:



Time-complexity is $O(c^n)$ i.e. Exponential time, where n is the number of vertices and c is the number of colors.

4. Hamiltonian Circuit Problem

Hamiltonian Path in an undirected graph is a path that visits each vertex exactly once. A Hamiltonian cycle (or Hamiltonian circuit) is a Hamiltonian Path such that there is an edge (in the graph) from the last vertex to the first vertex of the Hamiltonian Path. Determine whether a given graph contains Hamiltonian Cycle or not.

Given a graph G = (V, E) we have to find the Hamiltonian Circuit using Backtracking approach. We start our search from any arbitrary vertex say 'a.' This vertex 'a' becomes the root of our implicit tree. The first element of our partial solution is the first intermediate vertex of the Hamiltonian Cycle that is to be constructed. The next adjacent vertex is selected by alphabetical order. If at any stage any arbitrary vertex makes a cycle with any vertex other than vertex 'a' then we say that **dead end** is reached. In this case, we backtrack one step, and again the search begins by selecting another vertex and backtrack the element from the partial; solution must be removed. The search using backtracking is successful if a Hamiltonian Cycle is obtained.

Example1: Without loss of generality, we can assume that if a Hamiltonian circuit exists, it starts at vertex *a.* Accordingly, we make vertex *a* the root of the state-space tree (Figure 12.3b). The first component of our future solution, if it exists, is a first intermediate vertex of a Hamiltonian circuit to be constructed. Using the alphabet order to break the three-way tie among the vertices adjacent to *a,* we select vertex *b*. From *b*, the algorithm proceeds to *c*, then to *d*, then to *e*, and finally to *f,* which proves to be a dead end. So the algorithm backtracks from *f* to *e*, then to *d*, and then to *c*, which provides the first alternative for the algorithm to pursue. Going from *c* to *e* eventually proves useless, and the algorithm has to backtrack from *e* to *c* and then to *b*. From there, it goes to the vertices *f* , *e*, *c*, and *d*, from which it can legitimately return to *a*, yielding the Hamiltonian circuit ***a-b-f-e-c-d-a.*** If we wanted to find another Hamiltonian circuit, we could continue this process by backtracking from the leaf of the solution found.

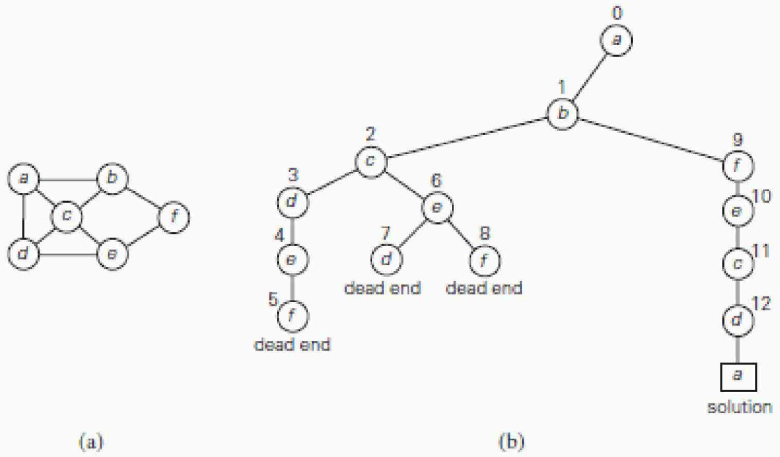
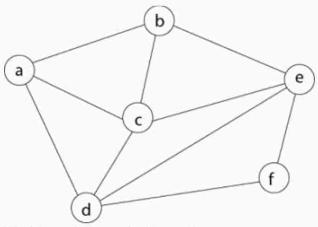


FIGURE 12.3 (a) Graph. (b) State-space tree for finding a Hamiltonian circuit. The numbers above the nodes of the tree indicate the order in which the nodes are generated.

Time-complexity is $O(n^n)$ i.e. Exponential time, where n is the number of vertices

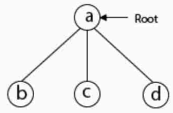
Example2: Consider a graph G = (V, E) shown in fig. we have to find a Hamiltonian circuit using Backtracking method.



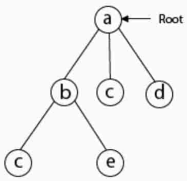
Solution: Firstly, we start our search with vertex 'a.' this vertex 'a' becomes the root of our implicit tree.



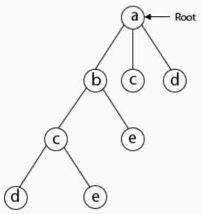
Next, we choose vertex 'b' adjacent to 'a' as it comes first in lexicographical order (b, c, d).



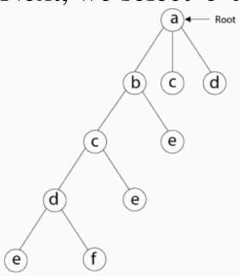
Next, we select 'c' adjacent to 'b.'



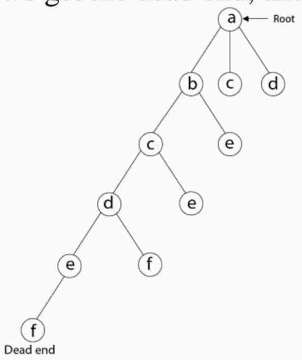
Next, we select 'd' adjacent to 'c.'



Next, we select 'e' adjacent to 'd.'

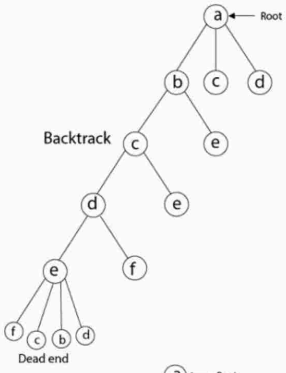


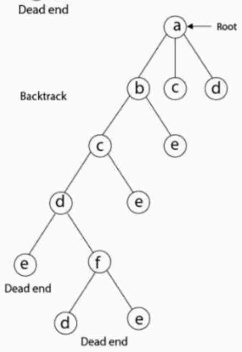
Next, we select vertex 'f' adjacent to 'e.' The vertex adjacent to 'f' is d and e, but they have already visited. Thus, we get the dead end, and we backtrack one step and remove the vertex 'f' from partial solution.



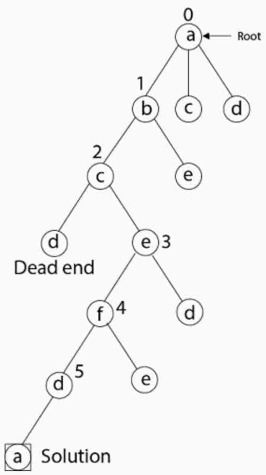
From backtracking, the vertex adjacent to 'e' is b, c, d, and f from which vertex 'f' has already been checked, and b, c, d have already visited. So, again we backtrack one step. Now, the vertex adjacent to d are e, f from which e has already been checked, and adjacent of 'f' are d and e. If 'e' vertex, revisited them we get a dead state. So again we backtrack one step.

Now, adjacent to c is 'e' and adjacent to 'e' is 'f' and adjacent to 'f' is 'd' and adjacent to 'd' is 'a.' Here, we get the Hamiltonian Cycle as all the vertex other than the start vertex 'a' is visited only once. (a - b - c - e - f -d - a).



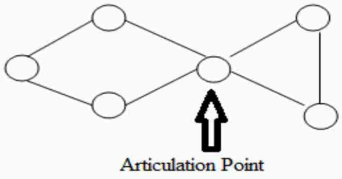


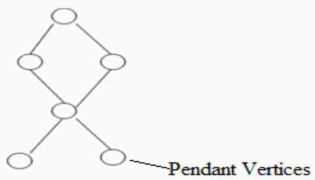
Again Backtrack



Here we have generated one Hamiltonian circuit, but another Hamiltonian circuit can also be obtained by considering another vertex.

Note: Hamiltonian cycles are not existing if graph contains an Articulation point or Pendant vertices.





Branch and Bound

An **optimization problem** seeks to Minimize or Maximize some objective function, usually subject to some constraints. In the terminology of optimization problems, a ***Feasible solution*** is a point in the problem's search space that satisfies all the problem's constraints, whereas an ***Optimal solution*** is a feasible solution with the best value of the objective function.

Compared to backtracking, branch-and-bound requires two additional items:

- For every node of a state-space tree, a bound on the best value of the objective function on any solution
- The value of the best solution seen so far.

If this information is available, we can compare a node's bound value with the value of the best solution seen so far. If the bound value is not better than the value of the best solution seen so far—i.e., not smaller for a minimization problem and not larger for a maximization problem—the node is non-promising (or not feasible) and can be terminated (or "pruned"). This is the principal idea of the branch-and-bound technique.

In general, Branch and Bound Technique follows two steps:

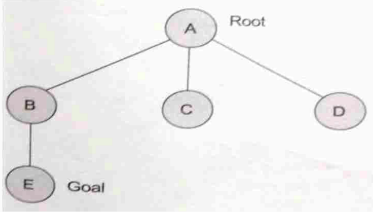
1. Branching - Involves division of a given problem into two or more sub-problems. These sub-problems are independent and exclusive, similar to the original problem but smaller in size.
2. Bounding – Which aids in limiting the growth of the State space tree. In this step, the best solution of the sub-problems is identified and used as a Lower bound / Upper Bound of the given problem. A Lower bound / Upper Bound is the optimistic estimate of the best solution. At every node in the State space tree, the bound is calculated and the node with the best bound will be selected to explore further. Any node which doesn't satisfy the Lower bound / Upper Bound will be cut off (Pruned).

We terminate a search path at the current node in a state-space tree for any one of the following three reasons:

- The value of the node's bound is not better than the value of the best solution seen so far.
- The node represents no feasible solutions because the constraints of the problem are already violated.
- The subset of feasible solutions represented by the node - In this case, we compare the value of the objective function for this feasible solution with that of the best solution seen so far and update if the new solution is better.

Popular search techniques for Branch and Bound technique are:

1. **First In First Out Branch and Bound (FIFOBB)** - FIFOBB uses Breadth first search(BFS) technique and Queue data structure. Therefore it is called as FIFO Branch and Bound. A BFS processes a sibling node earlier than the children nodes as shown in the figure below.



Initially node A is a root node and node E is the goal node also known as the solution node. Initially root node A is put into the queue. Then remove A and put it's children into the queue as shown in the below figure.

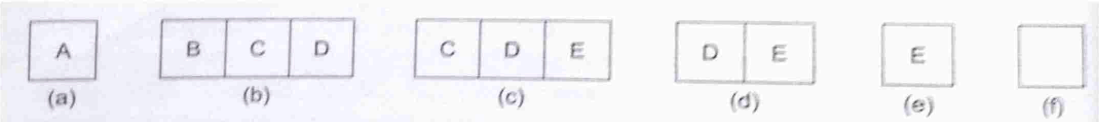


Fig. 15.2 Status of queue (a) Initial queue with root A (b) Enqueue successors of node A (c) Process B and its enqueue successors (d) Process C (e) Process D (f) Process E and terminate

Then delete node B from the queue and make it an E-node. Add all it's children to queue. If nodes are not promising, then it's children are not considered at all. They are called D**ead nodes.** The children of the Dead node are also dead nodes. Similarly node D is not productive and has no child. Therefore delete D. Now node E becomes an E-node. Since node E is a solution node, the algorithm terminates. In the branch and bound technique, a node is added to the queue only if it's bound is less than the current best solution. Otherwise, the node is discarded.

The algorithm for BFS based Branch and Bound algorithm(FIFOBB) is as below:
Step 1 : Initialize queue Q.
Step 2 : Let v be the root of a tree T.
Step 3 : Let the current value of v be "best".
Step 4 : Insert v into queue Q
Step 5 : While (Q is not empty) do the following:
5a : Remove node v
5b : For each child u of v do
If bound(u) is better than "best" then
Insert u into Q
Update "best" value as value(u)
Step 6 : End

2. **Last In First Out Branch and Bound (LIFOBB)** – DFS with Branch and Bound is a stack based approach and is called LIFOBB. It is similar to FIFOBB, except that it uses a Stack instead of a Queue. The informal algorithm for LIFOBB is similar to above algorithm, with a Stack replacing the Queue.

3. **Least Cost Branch and Bound (LCBB)** – Unlike BFS and DFS, an LC algorithm is an intelligent search technique. This also known as **Best First Search** technique. BFS and DFS are blind search algorithms which are ineffective and involve the selection of e-nodes that are sequential, and all the children of the previous node are generated before any other live node. Hence, a lot of time is spent on searching. By using LC algorithm, search time is reduced. This algorithm uses a cost function which varies from problem to problem. This method assigns a rank or priority to each node based on the cost function (also known as Bound function).

   The implementation of LCBB requires a Priority queue. The priority queue is a Max-heap for Maximization problems and Min-heap for Minimization problems. The informal algorithm is given below:
   Step 1 : Initialize Priority queue Q
   Step 2 : Let v be the root of a State space tree
   Step 3 : Let the current value of v be "best".
   Step 4 : Insert v into queue Q

Step 5 : Repeat the following steps till termination:

5a : Remove node v

5b : If node v is a solution node or goal node, then report solution; otherwise, if Q is empty then no solution is possible; generate a node and add it to the priority queue.

Step 6 : End

Knapsack Problem using LCBB

Given n items of known weights wi and values vi , $i = 1, 2, \ldots, n,$ and a knapsack of capacity W, find the most valuable subset of the items that fit in the knapsack. It is convenient to order the items of a given instance in descending order by their value-to-weight ratios. Then the first item gives the best payoff per weight unit and the last one gives the worst payoff per weight unit:

$$v_1/w_1 \geq v_2/w_2 \geq \cdots \geq v_n/w_n.$$

It is natural to structure the state-space tree for this problem as a binary tree constructed as follows:

1. Each node on the ith level of this tree, $0 \leq i \leq n,$ represents all the subsets of n items that include a particular selection made from the first i ordered items.
2. This particular selection is uniquely determined by the path from the root to the node: a branch going to the left indicates the inclusion of the next item, and a branch going to the right indicates its exclusion.
3. We record the total weight w and the total value v of this selection in the node, along with some upper bound ub on the value of any subset that can be obtained by adding zero or more items to this selection.

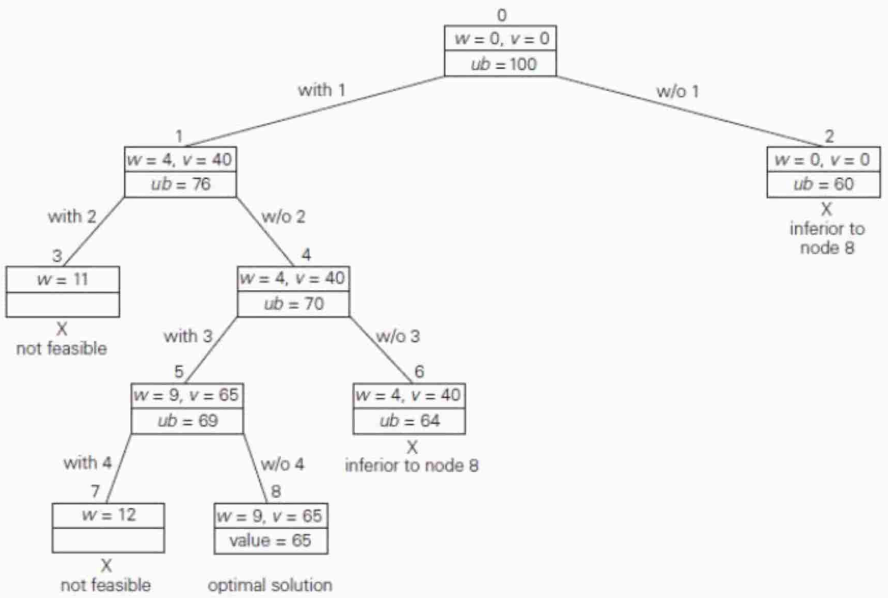
A simple way to compute the upper bound ub is to add to v, the total value of the items already selected, the product of the remaining capacity of the knapsack $W - w$ and the best per unit payoff among the remaining items, which is $vi+1/wi+1$:

$$ub = v + (W - w)(v_{i+1}/w_{i+1}).$$

Example:

item	weight	value	value / weight
1	4	$40	10
2	7	$42	6
3	5	$25	5
4	3	$12	4

The knapsack's capacity W is 10.



At the root of the state-space tree, no items have been selected as yet. Hence, both the total weight of the items already selected w and their total value v are equal to 0. The value of the upper bound computed by formula ub above is \$100.

Node 1, the left child of the root, represents the subsets that include item 1. The total weight and value of the items already included are 4 and \$40, respectively; the value of the upper bound is $40 + (10 - 4) * 6 = \$76$.

Node 2 represents the subsets that do not include item 1. So, $w = 0$, $v = \$0$, and $ub = 0 + (10 - 0) * 6 = \$60$. Since node 1 has a larger upper bound than the upper bound of node 2, it is more promising for this maximization problem, and we branch from node 1 first. Its children—nodes 3 and 4—represent subsets with item 1 and with and without item 2, respectively. Since the total weight w of every subset represented by node 3 exceeds the knapsack's capacity, node 3 can be terminated immediately.

Node 4 has the same values of w and v as its parent; the upper bound ub is equal to $40 + (10 - 4) * 5 = \$70$. Selecting node 4, we get nodes 5 and 6 by respectively including and excluding item 3. The total weights and values as well as the upper bounds for these nodes are computed in the same way as for the preceding nodes. Branching from node 5 yields node 7, which represents no feasible solutions, and node 8, which represents just a single subset {1, 3} of value \$65. The remaining live nodes 2 and 6 have smaller upper-bound values than the value of the solution represented by node 8. Hence, both can be terminated making the subset {1, 3} of node 8 the optimal solution to the problem.

Traveling Salesman Problem using LCBB

To apply the branch-and-bound technique to traveling salesman problem, we need to find lower bound on tour lengths. One very simple lower bound can be obtained as follows. For each city i, $1 \le i \le n$, find the sum S_i of the distances from city i to the two nearest cities; compute the sum s of these n numbers, divide the result by 2, and round up the result to the nearest integer:

$$lb = \lceil s/2 \rceil$$

For example, for the instance in Figure, above formula yields
$lb = [(1+3) + (3+6) + (1+2) + (3+4) + (2+3)]/2 = 14$.

Moreover, for any subset of tours that must include particular edges of a given graph, we can modify lower bound accordingly. For example, for all the Hamiltonian circuits of the graph in Figure below that must include edge (a, d), we get the following lower bound by summing up the lengths of the two shortest edges incident with each of the vertices, with the required inclusion of edges (a, d) and (d, a):
$[(1+5) + (3+6) + (1+2) + (3+5) + (2+3)]/2 = 16$.

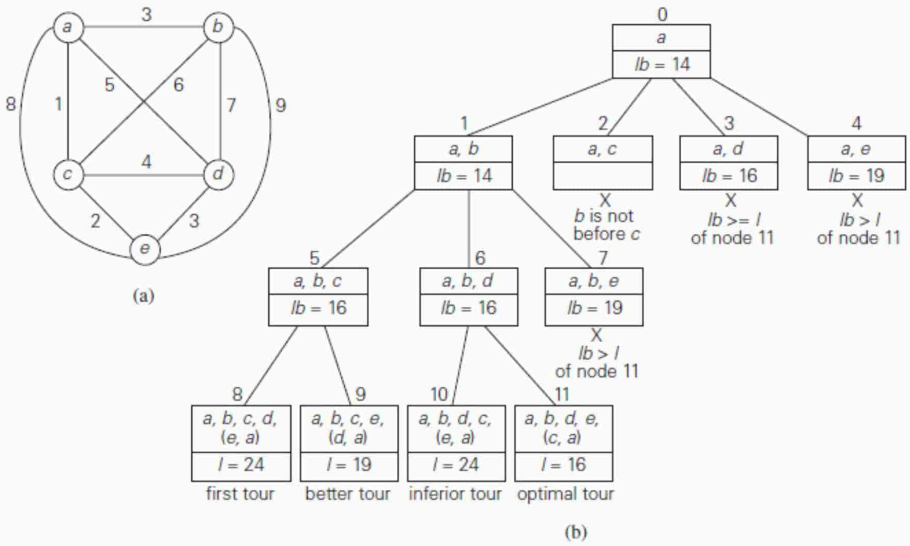


FIGURE 12.9 (a) Weighted graph. (b) State-space tree of the branch-and-bound algorithm to find a shortest Hamiltonian circuit in this graph. The list of vertices in a node specifies a beginning part of the Hamiltonian circuits represented by the node.

We now apply the branch-and-bound algorithm. To reduce the amount of potential work, we consider two assumptions:

1. We can consider only tours that start at *a*.
2. Because our graph is undirected, we can generate only tours in which *b* is visited before *c*.

In addition, after visiting $n - 1 = 4$ cities, a tour has no choice but to visit the remaining unvisited city and return to the starting one. The state-space tree tracing the algorithm's application is given in the above Figure.

The strengths and weaknesses of backtracking are applicable to branch-and-bound as well. The state-space tree techniques enable us to solve many large instances of difficult combinatorial problems. As a rule, however, it is virtually impossible to predict which instances will be solvable in a realistic amount of time and which will not. In contrast to backtracking, solving a problem by branch-and-bound has both the challenge and opportunity of choosing the order of node generation and finding a good bounding function. Though the Lease cost method or the Best-first rule we used above may or may not lead to a solution faster than other strategies.

Knapsack Problem using FIFOBB

0/1 Knapsack problem Using FIFO Branch and Bound.

$n = 4, \quad m = 15$

$(P_1\ P_2\ P_3\ P_4) = (10, 10, 12, 18)$

$(w_1\ w_2\ w_3\ w_4) = (2, 4, 6, 9)$

- Convert profits into negative values

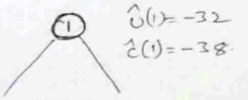
$(P_1\ P_2\ P_3\ P_4) = (-10, -10, -12, -18)$

- Calculate LB and UB for each node

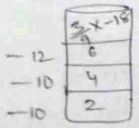
LB - Fractions are allowed

UB - Fractions are not allowed.

Global UB = -32

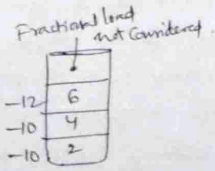


For node 1:



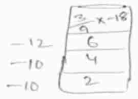
LB = -32 - 6 = -38

Fractional load not Considered.

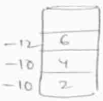


UB = -32

For node 2: $x_1 = 1$



LB = -32 - 6 = -38

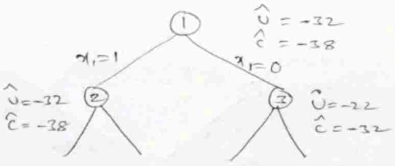


UB = -32

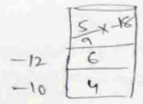
$(P_1\ P_2\ P_3\ P_4) = (-10, -10, -12, -18)$

$(w_1\ w_2\ w_3\ w_4) = (2, 4, 6, 9)$

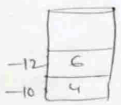
Global UB = -32



For node 3: $x_1 = 0$



LB = -22 - 10 = -32



UB = -22

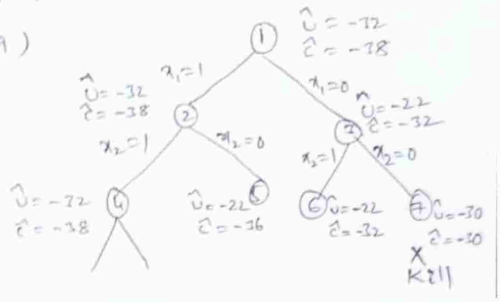
Note: Condition Checking for each node

1. New upper bound (UB) is compared with Global UB,
   a) If new UB is smaller than Global UB then update Global UB with new UB.
   b) If new UB and Global UB is same then no need to update.
2. If new LB value is greater than Global UB then Kill that node ie. no need to expand further.

$(P_1, P_2, P_3, P_4) = (-10, -10, -12, -18)$

$(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$

Global UB = -32

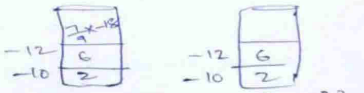


For node 4: $x_1 = 1, x_2 = 1$



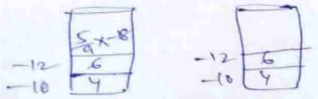
LB = -32 - 6 = -38 UB = -32

For node 5: $x_1 = 1, x_2 = 0$



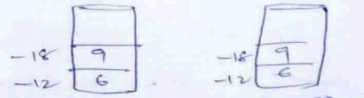
LB = -22 - 14 = -36 UB = -22

For node 6: $x_1 = 0, x_2 = 1$



LB = -22 - 10 = -32 UB = -22

For node 7: $x_1 = 0, x_2 = 0$



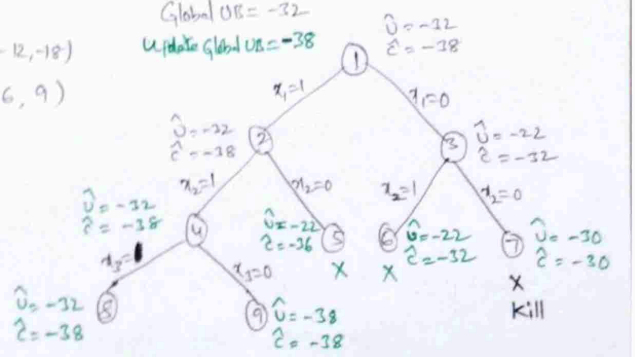
UB = -30

$(P_1, P_2, P_3, P_4) = (-10, -10, -12, -18)$

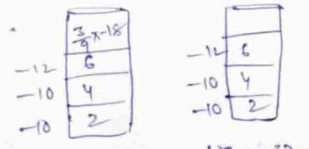
$(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$

Global UB = -32

Update Global UB = -38

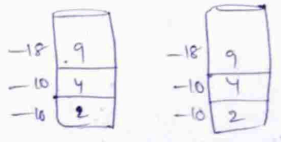


For node 8: $x_1 = 1, x_2 = 1, x_3 = 1$

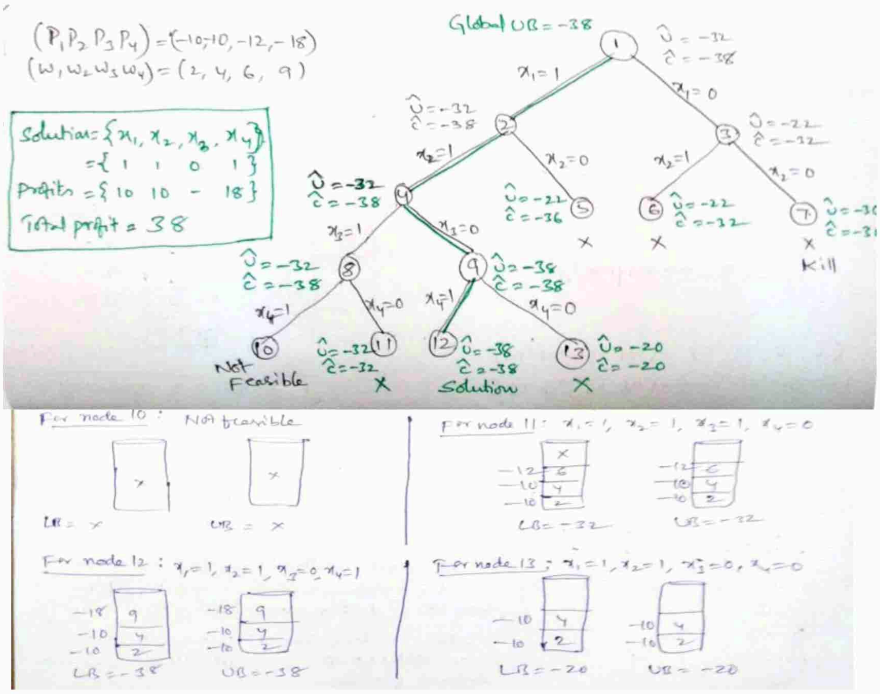


LB = -32 - 6 = -38 UB = -32

For node 9: $x_1 = 1, x_2 = 1, x_3 = 0$

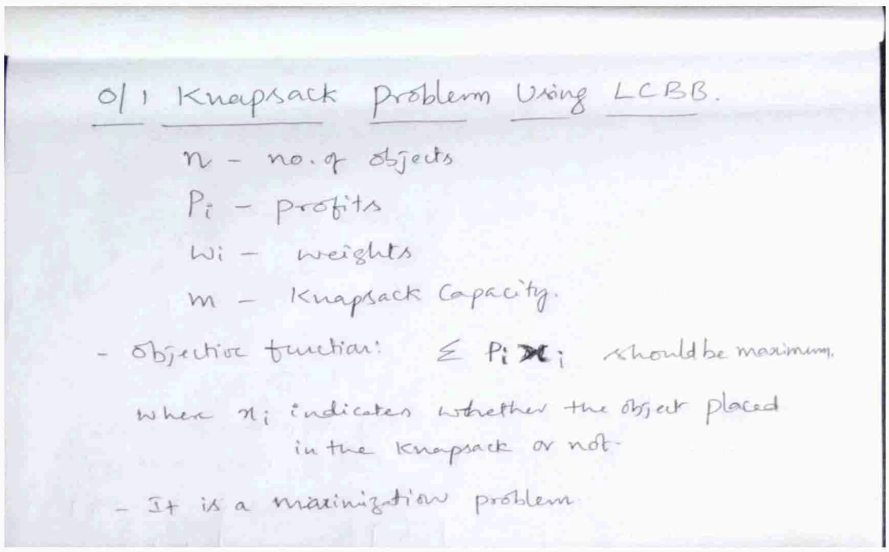


LB = -38 UB = -38



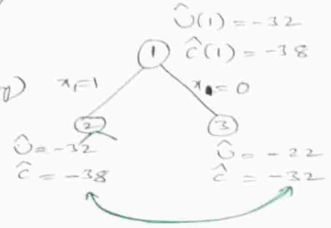
Alternative Methods:

0/1 Knap sack problem using LCBB

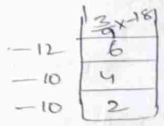


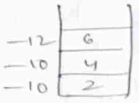
$(P_1, P_2, P_3, P_4) = (-10, -10, -12, -18)$

$(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$



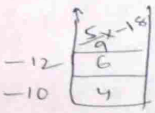
For node 2: $x_1 = 1$ (First object is considered)

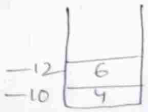




LB = −32 − 6 = −38 UB = −32

For node 3 $x_1 = 0$ (First object is not considered)



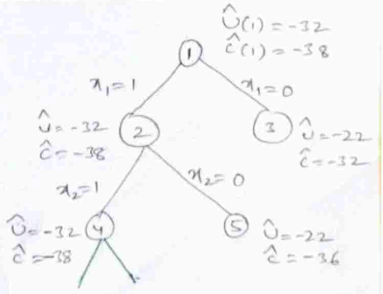


LB = −22 − 10 = −32 UB = −22

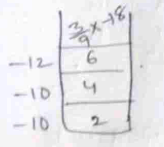
Compare node (2) and node (3) lower bound values and select smallest LB value, i.e. −38. So we explore children for node (2)

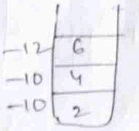
$(P_1, P_2, P_3, P_4) = (-10, -10, -12, -18)$

$(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$



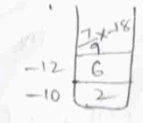
For node 4: $x_1 = 1, x_2 = 1$

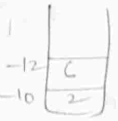




LB = −32 − 6 = −38 UB = −32

For node 5: $x_1 = 1, x_2 = 0$



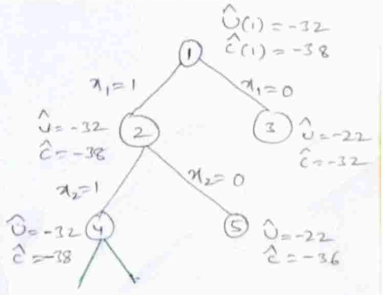


LB = −22 − 14 = −36 UB = −22

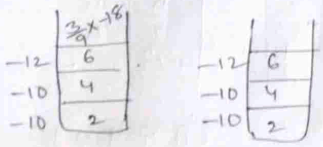
Now compare node (4) and node (5) LBs, Here node (4) LB is lower, so Consider node (4).

$(P_1, P_2, P_3, P_4) = (-10, -10, -12, -18)$

$(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$

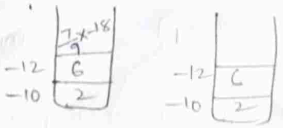


For node 4 : $x_1 = 1$, $x_2 = 1$



$LB = -32 - 6 = -38$ $\quad UB = -32$

For node 5 : $x_1 = 1$, $x_2 = 0$

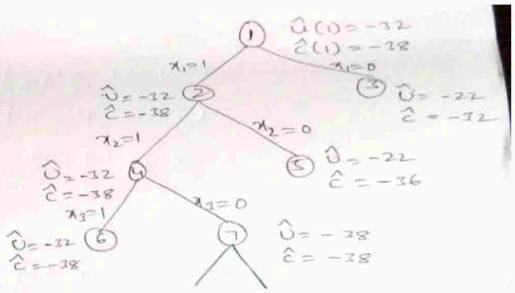


$LB = -22 - 14 = -36$ $\quad UB = -22$

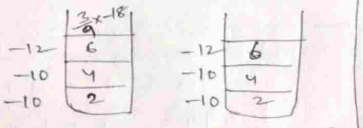
Now compare node (4) and node (5) LB's, Here node (4) LB is lower, so Consider node (4).

$(P_1, P_2, P_3, P_4) = (-10, -10, -12, -18)$

$(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$



For node 6: $x_1 = 1$, $x_2 = 1$, $x_3 = 1$



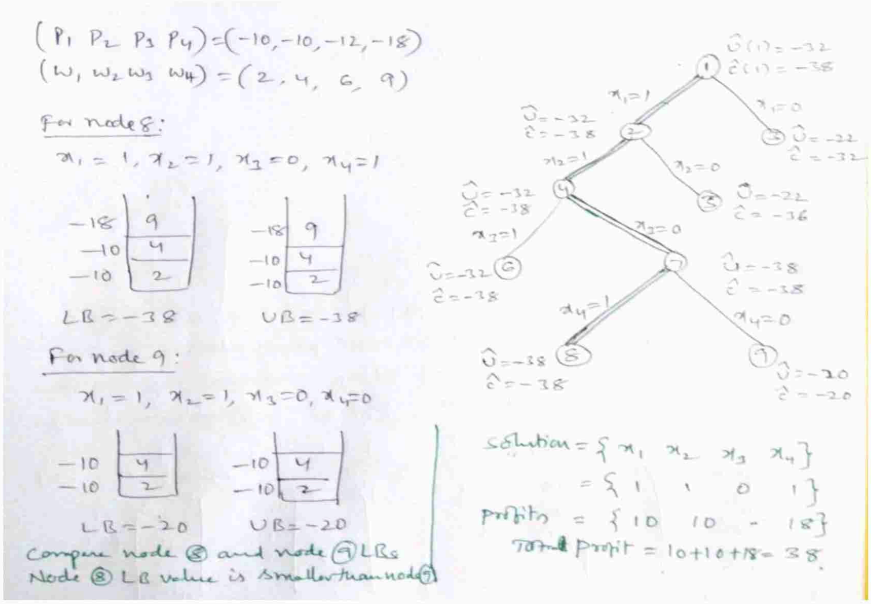
$LB = -32 - 6 = -38$ $\quad UB = -32$

For node 7 $x_1 = 1$, $x_2 = 1$, $x_3 = 0$

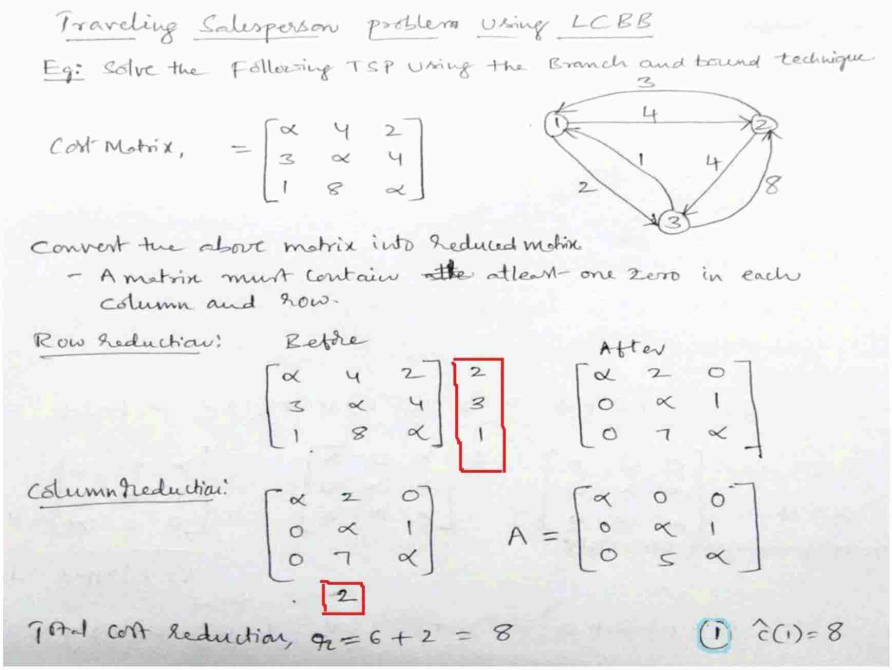


$LB = -38$ $\quad UB = -38$

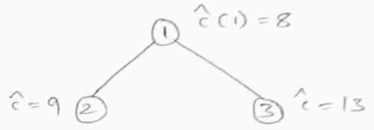
Compare node (6) and node (7) lower bounds. Here both are same then we compare upper bound values of node (6) and (7). Node (7) upper bound value is less than node (6). So select node (7) for exploring children.



Traveling salesperson problem using LCBB



To calculate path (i, j)
Replace all entries
Row i } ∞
Column j }
(j, i) ⟶ ∞



For path (1, 2)

$$A=\begin{bmatrix}\infty & 0 & 0\\ 0 & \infty & 1\\ 0 & 5 & \infty\end{bmatrix}$$

$$\begin{bmatrix}\infty & \infty & \infty\\ \infty & \infty & 1\\ 0 & \infty & \infty\end{bmatrix} \quad 1$$

After Row Reduction & Column Reduction

$$\begin{bmatrix}\infty & \infty & \infty\\ \infty & \infty & 0\\ 0 & \infty & \infty\end{bmatrix}$$

$r = 1$

$$\hat{c}(2) = \underset{\text{parent}}{\hat{c}(1)} + A(1,2) + r = 8 + 0 + 1 = 9$$

For Path (1, 3)

$$A=\begin{bmatrix}\infty & 0 & 0\\ 0 & \infty & 1\\ 0 & 5 & \infty\end{bmatrix}$$

$$\begin{bmatrix}\infty & \infty & \infty\\ 0 & \infty & \infty\\ \infty & 5 & \infty\end{bmatrix} \quad 5$$

After Row & Column Reduction

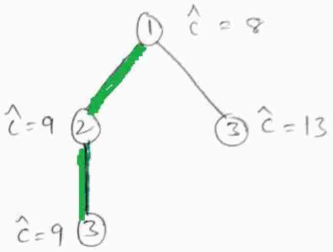
$$\begin{bmatrix}\infty & \infty & \infty\\ 0 & \infty & \infty\\ \infty & 0 & \infty\end{bmatrix}$$

$r = 5$

$$\hat{c}(3) = \hat{c}(1) + A(1,3) + r = 8 + 0 + 5 = 13$$

Now, A in the path that occurs at ① — ②

$$A=\begin{bmatrix}\infty & \infty & \infty\\ \infty & \infty & 0\\ 0 & \infty & \infty\end{bmatrix}$$



For Path (2, 3)

$$A=\begin{bmatrix}\infty & \infty & \infty\\ \infty & \infty & 0\\ 0 & \infty & \infty\end{bmatrix}$$

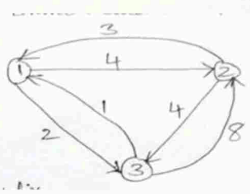
$$\begin{bmatrix}\infty & \infty & \infty\\ \infty & \infty & \infty\\ \infty & \infty & \infty\end{bmatrix}$$

After Row and Column Reduction

$r = 0$

$$\hat{c}(3) = \hat{c}(2) + A(2,3) + r$$
$$= 9 + 0 + 0 = 9$$

Solution: 1 − 2 − 3 − 1
4 + 4 + 1 = 9.



Backtracking technique	**Branch-and-Bound technique**
Uses only DFS technique	Is not limited to any search technique; BFS, DFS, or Least-cost search can be used
Backtracking finds the solution to the overall issue by finding a solution to the first subproblem and them recursively solving other subproblems based on the solution of the first issue.	Branch and bound solves a given problem by dividing it into at least two new restricted subproblems. Hence, this is another difference between backtracking and branch and bound.
Can be used for all sorts of problems such as decision, enumerative, and optimization problems	Can be used only for an optimization problems
More Efficient	Less Efficient